

Mono2Sim-GS: LingBot-Guided Gaussian Reconstruction from Monocular RGB Video

An RGB-First Edge-Cloud Pipeline for Pose-Guided Gaussian Scene Reconstruction

Yanheng Zhu, Yue Peng, Yongzhe Zhao
Computer Vision Final Project
NYU Shanghai

Abstract

Robots can collect monocular RGB video cheaply, but turning that video into a scene representation useful for simulation, navigation, and visual inspection remains difficult. A simulator-facing scene needs explicit geometry for collision, occupancy, and planning, while a camera-facing scene needs visually realistic rendering. These requirements are poorly served by a single representation: meshes and point clouds are geometrically useful but visually incomplete, while 3D Gaussian splats provide strong real-time rendering but are not directly simulation-ready. This project builds and evaluates Mono2Sim-GS, a calibration-light RGB-first reconstruction layer that combines low-latency edge tracking, streaming learned geometry, and feed-forward Gaussian visual reconstruction. The system uses cuVSLAM for low-latency live tracking, LingBot-Map as the geometry provider for estimated depth, intrinsics, and global poses, and GenWildSplat as a learned visual Gaussian prior. The main methodological contribution is LingBot-Guided Gaussian Splatting (LG-GS): instead of asking GenWildSplat to infer pose, depth, scale, and appearance from weakly-overlapping video keyframes, we inject LingBot depth, intrinsics, and poses to initialize Gaussian means in the LingBot world frame. We evaluate the design on a 10-second, 301-frame indoor RGB video and a live HikRobot/cuVSLAM/LingBot pipeline. The results show that local GenWildSplat chunks can produce sharp views but fail global continuity; single global 32–44 context reconstructions increase coverage but become blurry and memory-heavy; and LG-GS improves pose synchronization and coordinate-frame consistency while final visual quality still requires local photometric Gaussian optimization.

1 Introduction

Monocular RGB video is easy to collect on phones, webcams, and robot cameras, but it is not directly useful for simulation or embodied reasoning. Downstream systems need camera motion, persistent geometry, visually realistic rendering, and eventually physical assets for navigation or simulator export. This report studies a calibration-light setting: the input is an uncalibrated or weakly calibrated RGB stream rather than a manually calibrated RGB-D rig or dense offline photo collection. In this project, calibration-light means camera intrinsics and poses are estimated or provided by the learned/SLAM front-end rather than by a separately calibrated sensor rig. The project question is:

Can we turn a calibration-light monocular RGB video or robot RGB stream into a pose-aligned 3D visual scene layer that is useful as the first stage of a real-to-sim pipeline?

Mono2Sim-GS addresses this problem with an edge-cloud reconstruction pipeline. A live HikRobot/cuVSLAM path keeps RGB preview and low-latency tracking responsive, while LingBot-Map and GenWildSplat run asynchronously or offline to produce depth, colored point clouds, and Gaussian visual assets. The output is not yet a complete simulator scene; it is the reconstruction layer that would precede TSDF fusion, mesh cleanup, navigation-map export, and simulator integration.

A real-to-sim scene has two different jobs. The physical side must support collision checking, occupancy reasoning, local planning, and simulation geometry. For these tasks, explicit depth, TSDFs, meshes, or occupancy maps are appropriate. The visual side must produce realistic camera observations for inspection, policy evaluation, and future vision-language-action systems. For this task, neural rendering and 3D Gaussian Splatting are attractive because they render view-dependent appearance efficiently [1].

Using only one representation creates a mismatch. A mesh can be navigable but visually flat. A Gaussian scene can look realistic but may contain low-opacity floating artifacts, inconsistent scale, no watertight boundary, and no direct collision surface. We therefore use a hybrid design:

$$\begin{aligned} \text{geometry} &\rightarrow \{\text{depth, points, mesh, occupancy}\}, \\ \text{visual} &\rightarrow \{\text{3D Gaussians, RGB rendering}\}. \end{aligned} \quad (1)$$

1.1 Contributions

The system combines three complementary components:

- **cuVSLAM** provides low-latency visual odometry suitable for edge robotics deployments [2].
- **LingBot-Map** provides learned streaming geometry: camera poses, depth, and colored point clouds [3].
- **GenWildSplat** provides a fast feed-forward Gaussian prior from sparse, unposed images [4].

The live architecture follows a frontend/backend separation: cuVSLAM keeps the tracking path responsive, while LingBot dense geometry and GenWildSplat Gaussian generation run asynchronously or offline when more compute is available. In our implementation, cuVSLAM is the low-latency live tracker, while LingBot poses define the offline/global geometry frame used by LG-GS and by the WebUI replay camera. This lets the system expose current RGB and pose without waiting for every dense reconstruction update.

Our experiments show that GenWildSplat is strong as a sparse-view visual prior but is not, by itself, a robust long-video global mapper. In a 10-second video, early frames show a sofa/table region, while later frames move toward a hallway with weak overlap and low-texture walls. More keyframes alone do not make a feed-forward sparse-view model more coherent. Instead, they can force multiple weakly-overlapping local scenes into one prediction, causing blur and misalignment.

Our main method is LingBot-Guided Gaussian Splatting (LG-GS), a pose-guided GenWildSplat integration. Original GenWildSplat predicts camera pose, depth, and Gaussians from images:

$$I_{1:k} \rightarrow \{\hat{T}_i, \hat{D}_i, \hat{G}\}. \quad (2)$$

LG-GS changes the responsibility split:

$$I_{1:k}, K_i, T_i^{\text{LingBot}}, D_i^{\text{LingBot}} \rightarrow G_{\text{pose-guided}}. \quad (3)$$

LingBot supplies the geometric scaffold, GenWildSplat supplies visual Gaussian attributes, and the WebUI uses LingBot global poses rather than cuVSLAM poses to synchronize real RGB and

Gaussian rendering. In this project, we implement: (1) a functional edge-cloud reconstruction backbone, (2) a hybrid geometry/Gaussian real-to-sim representation, (3) the LG-GS pose-guided Gaussian initialization, and (4) an ablation-style study comparing raw chunks, chunk atlases, global keyframe reconstructions, preprocessing variants, display cleanup, and LG-GS variants.

2 Related Work

2.1 Classical Reconstruction, SLAM, and Volumetric Fusion

Classical 3D reconstruction systems estimate camera motion and scene geometry from multiple images. Structure-from-motion systems such as COLMAP recover camera poses and sparse/dense geometry from image collections [5]. Visual SLAM systems estimate camera motion online and often maintain local maps for tracking. For robot applications, the resulting poses and maps are valuable because they are connected to planning and control.

Volumetric fusion methods integrate many depth observations into an explicit surface representation. Curless and Levoy’s volumetric method [6] and KinectFusion [7] are canonical examples. These methods motivate the physical side of our pipeline: a simulator eventually needs a mesh, TSDF, or occupancy map, not only a renderable neural asset.

cuVSLAM is relevant because it targets real-time visual odometry on edge devices. It supports flexible visual-inertial sensor configurations and is CUDA-accelerated for Jetson-class deployments [2]. In our design, cuVSLAM motivates the fast pose path. However, pose alone does not provide dense geometry or photo-realistic rendering.

2.2 Neural Rendering and 3D Gaussian Splatting

Neural Radiance Fields showed that multi-view images can be optimized into a continuous scene representation for novel view synthesis [8]. 3D Gaussian Splatting later introduced an explicit radiance-field representation based on anisotropic Gaussians that supports high-quality real-time rendering [1]. Each Gaussian can be understood as a primitive with mean μ , covariance or scale/rotation, opacity α , and color or spherical-harmonic appearance.

Gaussian representations are attractive for our project because they can render visually plausible views in a browser or GPU viewer. They are also easier to inspect and stream than fully implicit radiance fields. Their limitation is that rendering quality does not imply simulation utility. A raw Gaussian cloud may include floating splats, holes, duplicate surfaces, and no explicit collision surface. Accordingly, our final target is a hybrid mesh-Gaussian scene, not a Gaussian-only simulator.

2.3 Gaussian SLAM and Dense Mapping

Recent systems such as SplatAM use 3D Gaussians for dense RGB-D SLAM, jointly tracking camera motion and mapping a Gaussian scene [9]. These methods show that Gaussian maps can support online reconstruction when depth is available and when tracking/mapping are optimized together. Our setting differs in two important ways. First, our main input is RGB-first monocular video or a robot RGB stream, so depth must come from a learned geometry front-end. Second, our goal is not only online neural mapping, but a real-to-sim pipeline that separates physical geometry from visual rendering. Unlike SplatAM, we do not optimize a Gaussian map online with RGB-D input; instead, we use LingBot monocular geometry as a scaffold and keep Gaussian reconstruction as an asynchronous visual asset path.

2.4 Streaming Learned Geometry

LingBot-Map is a feed-forward streaming 3D reconstruction model built around geometric context attention [3]. It is designed to recover camera poses and point clouds from long video streams while maintaining compact context. LingBot is a natural geometry provider for our project because it gives depth, confidence, poses, and colored point clouds from RGB video. In our pipeline, LingBot is not treated as a monolithic complete simulator. Instead, it is used as a geometry grounding layer and as the source of pose/depth guidance for LG-GS.

2.5 Feed-Forward Sparse-View Gaussian Reconstruction

GenWildSplat reconstructs 3D scenes from sparse, unposed images using a feed-forward model [4]. Its strength is speed and generalization: it can predict depth, camera parameters, and 3D Gaussians without per-scene optimization. The GenWildSplat paper also uses SyncFix as a visualization-oriented multi-view refinement step [10]. However, SyncFix refines rendered views; it does not directly make a raw 3D Gaussian asset simulation-ready.

Our experiments show that GenWildSplat’s official sparse-view setting is different from long-sequence robot video. A few high-overlap views can produce a good local scene, but a 301-frame sequence with weak overlap requires global geometry management. LG-GS is our attempt to reuse GenWildSplat where it is strong – visual Gaussian prediction – while externalizing pose and depth to LingBot.

2.6 Real-Time Embodied Systems

Latency-aware embodied AI systems highlight that robot perception and control often run at different rates [11]. This motivates our architectural choice: fast pose and RGB preview should not block on slower dense geometry or cloud-side Gaussian generation. For a robot acting in a changing environment, a timely partial state can be more useful than a delayed complete reconstruction.

3 Methods

3.1 System Overview

The system is organized as an edge-cloud reconstruction pipeline. Fast components stay on the edge or near the camera; heavy reconstruction can run offline on a workstation GPU.

3.2 Workload Split

Different outputs impose different latency requirements. The RGB preview is the operator’s immediate feedback and should update at the camera rate. Pose must remain low-latency because downstream robot actions depend on current camera motion. Dense geometry can lag by several seconds if the latest available estimate is clearly labeled and updated asynchronously. Gaussian reconstruction is the heaviest component and is treated as an offline/cloud-side visual asset generation path.

Formally, the system should not require a single synchronized rate:

$$\text{RGB fps} = \text{geometry fps} = \text{Gaussian fps}. \quad (4)$$

Instead, the architecture uses separate rates:

$$\text{RGB preview} \gg \text{geometry update} \geq \text{Gaussian update}. \quad (5)$$

This separation is important for edge robotics: the perception loop should continue publishing RGB and pose even while dense reconstruction is delayed.

Mono2Sim-GS System Overview

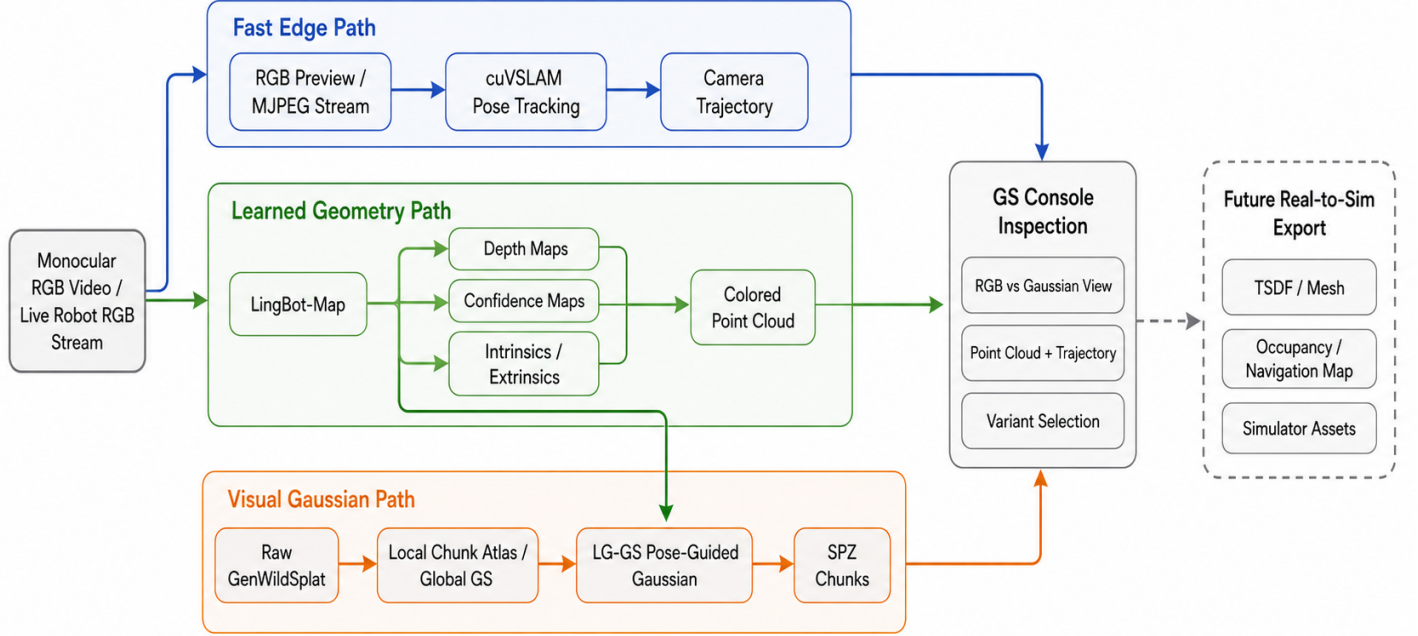


Figure 1: High-level system decomposition. The fast edge path keeps RGB and live tracking responsive, LingBot supplies the offline/global geometry frame and pose/depth guidance, LG-GS produces pose-guided Gaussian assets, and simulator export remains future work.

3.3 LingBot Geometry Front-End

LingBot provides estimated camera intrinsics K_i , camera-to-world poses $T_i = (R_i, t_i) \in \text{SE}(3)$, depth maps D_i , confidence maps C_i , and colored point-cloud outputs. Given a pixel $u = (x, y)$, homogeneous pixel coordinate $\tilde{u} = [x, y, 1]^T$, and depth $D_i(u)$, we back-project to camera coordinates:

$$X_c(u) = D_i(u)K_i^{-1}\tilde{u}. \quad (6)$$

Then the point is transformed into the LingBot world frame:

$$X_w(u) = R_i X_c(u) + t_i. \quad (7)$$

These world-space points are used in two ways. First, they create the colored point-cloud visualization. Second, they become the geometry prior for LG-GS.

3.4 Raw GenWildSplat Route

The first visual path used GenWildSplat directly. We selected video keyframes, generated Gaussian PLY and attribute outputs, and converted them into WebUI-friendly streamable chunks. This route verified that GenWildSplat could produce useful local visual assets, especially in the sofa/table region. The problem was coordinate gauge. GenWildSplat predicts a scene in its own arbitrary coordinate frame; LingBot and the WebUI use a different world frame. Without alignment, the RGB stream and Gaussian render do not describe the same camera trajectory.

3.5 Sim(3) Alignment for Raw Gaussians

For raw GenWildSplat outputs, we aligned GenWildSplat camera centers or point-cloud centers to LingBot camera centers using a similarity transform:

$$\min_{s, R, t} \sum_i \|s R p_i + t - q_i\|_2^2, \quad (8)$$

where p_i are GenWildSplat camera centers, q_i are LingBot camera centers, s is scale, $R \in \text{SO}(3)$, and $t \in \mathbb{R}^3$. This improved local alignment, but one global Sim(3) cannot fix local drift across weak-overlap chunks. It also cannot repair incorrect topology inside the Gaussian scene.

3.6 Chunk Atlas

To preserve local visual quality, we reconstructed temporal chunks independently. The initial non-overlap chunks were:

$$[0, 100], [100, 200], [200, 300]. \quad (9)$$

We then used overlapping chunks:

$$[0, 120], [60, 180], [120, 240], [180, 300]. \quad (10)$$

Each chunk was reconstructed, aligned, converted to SPZ chunks, and registered in the WebUI manifest. The UI selected the active Gaussian variant based on playback frame and cross-faded in overlap regions. This helped inspection, but it did not solve global scene continuity. Cross-fading hides a hard visual switch; it does not jointly optimize two Gaussian scenes into one coherent asset.

3.7 LG-GS: Pose-Guided GenWildSplat

LG-GS is the central method contribution. Instead of using GenWildSplat as a fully pose-free system, we inject external LingBot predictions into the Gaussian export path: estimated intrinsics, camera poses, depth maps, confidence maps, and sampled world-space points. In the current implementation, this is a post-prediction/export-time modification: we keep GenWildSplat-predicted appearance-related attributes and overwrite or initialize Gaussian centers using LingBot-unprojected world points.

For selected keyframes, LingBot depth and camera geometry define Gaussian centers:

$$\mu_{i,u}^{\text{LG-GS}} = R_i^{\text{LingBot}} \left(D_i^{\text{LingBot}}(u) K_i^{-1} \tilde{u} \right) + t_i^{\text{LingBot}}. \quad (11)$$

GenWildSplat still predicts appearance-related attributes such as color, opacity, scale, and rotation. The responsibility split is:

$$\text{LingBot: pose, depth, geometric frame,} \quad (12)$$

$$\text{GenWildSplat: visual Gaussian attributes.} \quad (13)$$

3.8 Algorithmic Summary

Input: RGB frames $I_{1:T}$, selected keyframes $\mathcal{K}$, LingBot predictions $\{K_i, T_i, D_i, C_i\}$.
Output: Pose-guided Gaussian visual asset $G_{\text{LG-GS}}$.

1. Run LingBot on the RGB sequence to obtain depth, confidence, intrinsics, extrinsics, and colored points.
2. Select $k = 12$ keyframes spanning the 301-frame video.
3. For each keyframe and sampled pixel, back-project using Eq. 11.
4. Inject the resulting world-space points as Gaussian means in the GenWildSplat export path while keeping GenWildSplat-predicted visual attributes.
5. Export Gaussian attributes, convert PLY to SPZ chunks, and register the variant in the WebUI manifest.
6. During playback, use LingBot global camera pose for the Gaussian render camera, so RGB and GS views share the same trajectory.

Figure 2: Practical LG-GS procedure.

3.9 WebUI Inspection Console

The GS Console UI was extended from a live monitor into a reconstruction inspection console. It includes:

- a central colored point cloud and trajectory view,
- a current scan or rolling local cloud panel,
- an RGB-vs-Gaussian comparison panel,
- a NAV2-style 2D projection/debug map,
- reset playback control,
- Gaussian variant selection and chunk caching.

The transport path was also changed. RGB frames use an MJPEG fast path instead of large JSON messages. Current and global point clouds use binary WebSockets. Gaussian PLY files are converted to SPZ chunks and cached. These changes are necessary because early UI bottlenecks can otherwise be mistaken for reconstruction failures even when the underlying issue is data transport.

3.10 Display Calibration and Cleanup

Raw Gaussian assets frequently contain many low-opacity splats and very small scale axes. In a black-background renderer, this can appear as black speckles, grid-like holes, or overly sparse walls. We tested display-level processing:

- scale inflation,
- minimum linear scale clamping,
- opacity gamma adjustment,
- dropping very low opacity splats,
- dropping dark low-opacity splats.

These operations improve readability, but they are not true reconstruction. Increasing scale fills holes but blurs edges. Dropping splats removes noise but may remove valid surfaces. The correct next step is photometric Gaussian optimization initialized by LG-GS.

3.11 Future Local Photometric Optimization

Given a LG-GS Gaussian initialization G , LingBot poses, intrinsics, and RGB frames, a future local photometric optimization stage can refine visual quality:

$$\mathcal{L} = \lambda_1 \|R(G, K_i, T_i) - I_i\|_1 + \lambda_{\text{ssim}} (1 - \text{SSIM}(R_i, I_i)) + \lambda_s \mathcal{L}_{\text{scale}} + \lambda_o \mathcal{L}_{\text{opacity}} + \lambda_\Delta \|\Delta\mu\|_2^2. \quad (14)$$

Here $R(G, K_i, T_i)$ is the rendered view. The last term keeps optimized Gaussian means close to the LingBot-guided initialization. We expect this stage to be local/windowed rather than one global optimization over the entire long video.

4 Data and Experiments

4.1 Experimental Inputs and Setup

The main offline sequence is a ~ 10 -second, 301-frame indoor RGB video. Early frames observe a sofa/table region; later frames move toward a hallway and low-texture wall. This makes the sequence useful for testing long-video reconstruction because local windows have reasonable overlap, while the full trajectory contains weakly connected regions that stress global scale, pose, and appearance consistency.

LingBot processes the sequence with a 518-wide input setting and produces depth maps of approximately 294×518 , depth confidence, intrinsics, extrinsics, and colored point-cloud outputs. The live validation uses a HikRobot RGB camera, ROS topics, cuVSLAM tracking, asynchronous LingBot dense updates, and the GS Console WebUI. We also compare three preprocessing choices: a default square crop, a wide 518×294 LingBot-aligned crop, and a letterbox version that preserves field of view while padding to a square input.

4.2 Evaluation Questions

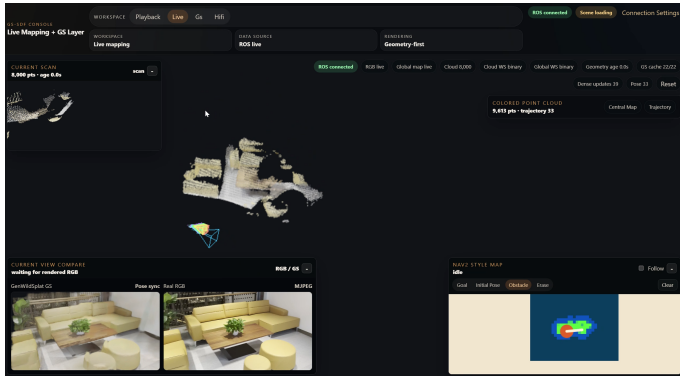
The experiments answer five questions:

1. Can the live edge pipeline stream RGB, pose, and dense geometry without blocking?
2. Does LingBot provide usable full-video geometry for point-cloud visualization and Gaussian guidance?
3. Do raw GenWildSplat chunks work as local visual assets?
4. Does increasing the number of global keyframes solve long-video reconstruction?
5. Does LG-GS improve pose synchronization and coordinate-frame consistency compared with raw pose-free Gaussians?

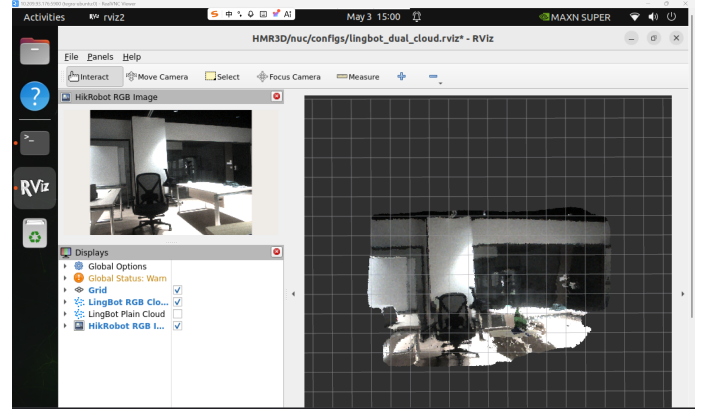
The evaluation is mostly qualitative and systems-oriented because the project goal is reconstruction integration rather than a benchmark model. We record point counts, latency, chunk statistics, keyframe settings, splat counts, alignment errors, memory limits, and visual failure modes observed in the WebUI.

4.3 Experiment 1: Live Edge Pipeline

Table 1 summarizes the live HikRobot/cuVSLAM/LingBot/GS Console validation run. The main finding is that RGB preview and tracking remain usable for operator inspection while dense geometry updates asynchronously.



(a) RGB-vs-Gaussian inspection.



(b) Live RGB, point cloud, and reconstruction console.

Figure 3: GS Console views used for qualitative inspection and failure analysis.

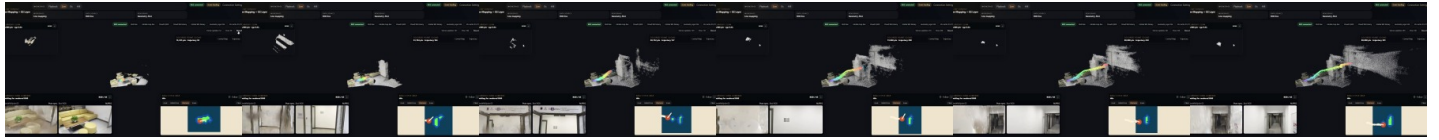


Figure 4: Inspection workflow used in the experiments. Left to right: RGB input, point-cloud state, Gaussian visualization, and UI-level comparison.

Table 1: Live edge pipeline observations. RGB preview and dense geometry update at different rates; the table keeps only the main prototype-side indicators.

Metric	Observed value
HikRobot RGB pre-view rate	4–5 fps
Rolling map point count	49,753 points
Dense updates processed	7
Trajectory length	103 poses
Saved RGB keyframes	73
Queue wait mean / max	0.011 s / 0.038 s
Worker end-to-end mean / median	5.14 s / 4.09 s
Geometry age mean / median	5.59 s / 4.47 s
Later dense model forward	1.9–3.3 s after warmup

The first dense window has a warmup cost of about 11.54 s, but later windows are faster. No worker failures or queue drops were observed in this run. The system therefore behaves as a latest-available geometry pipeline rather than a synchronized full-rate mapping pipeline.

4.4 Experiment 2: Full-Video LingBot Geometry

The offline 301-frame video was processed with LingBot at a 518-wide input setting. The result produced depth, confidence, intrinsics, extrinsics, and colored point outputs. This experiment corrected an early failure mode: a synthetic or low-resolution depth fallback produced a misaligned WebUI point cloud even though the UI pipeline was functioning. Using real LingBot depth and poses produced a more coherent point cloud and trajectory.

Table 2: Offline LingBot video geometry setup.

Item	Value
Video length	~ 10 seconds
Frames	301
LingBot input setting	518-wide baseline
Depth resolution	~ 294 × 518
Outputs	depth, confidence, intrinsics, extrinsics, colored points
Use in project	geometry visualization and LG-GS prior

4.5 Experiment 3: Raw Local GenWildSplat Chunks

The first GenWildSplat attempts used a small number of context views. The first sofa/table chunk produced the highest local visual quality, showing that GenWildSplat works well as a sparse-view visual prior. However, during full playback the Gaussian view could remain visually tied to the early sofa/table region while the RGB stream moved into the hallway. This exposed two issues:

- sparse local reconstruction does not cover the whole video,
- GenWildSplat’s predicted gauge does not match LingBot’s world frame.

4.6 Experiment 4: Overlap Chunk Atlas

We then built an overlapping chunk atlas. Table 3 shows the recorded statistics.

Table 3: Overlap chunk atlas. Lower alignment error is better. Alignment error is the mean Euclidean residual between Sim(3)-aligned GenWildSplat camera centers and LingBot camera centers, reported in normalized trajectory-scale units. The late hallway chunk has substantially higher alignment error.

Variant	Splats	Align.	Observation
chunk000_120	1,039,068	0.0505	highest sofa/table local quality
chunk060_180	878,408	0.0563	moderate transition quality
chunk120_240	589,799	0.0889	lower hallway and transition quality
chunk180_300	836,546	0.2229	highest late-video alignment error and lowest visual quality

The chunk atlas made the best local results accessible in the WebUI, but it did not create a smooth global Gaussian scene. Independent chunks had different scale, opacity, pose, and local geometry. Cross-fade reduced hard switching but could not eliminate duplicate surfaces or inconsistent view-dependent appearance.

4.7 Experiment 5: Single Global Keyframe Count

We then tried to reconstruct the full 10-second video as one global Gaussian with 32–48 context views. Table 4 summarizes the main variants.

The result was counterintuitive but consistent: more keyframes improved coverage but did not guarantee better quality. When context views span weakly-overlapping regions, the model must explain multiple local scenes in one feed-forward pass. The output became more complete but blurrier, and display-side refinement could not fully repair incorrect pose or topology. Thus, additional frames alone are insufficient for long-video feed-forward Gaussian reconstruction.

Table 4: Global GenWildSplat context ablation. More context increases coverage but does not monotonically improve reconstruction quality. The ‘refined’ variant applies display-side cleanup only; it is not the future local photometric optimization discussed in Sec. 3.11.

Variant	Views	Status	Observation
global_32ctx	32	done	better coverage than local chunks; visible blur
global_40ctx	40	done	increased coverage; cross-region ambiguity remained
global_44ctx	44	done	sharper display setting; topology errors remained
global_44ctx_refined	44	done	display refinement improved local appearance; global errors remained
global_48ctx	48	failed	exceeded available memory budget

4.8 Experiment 6: LG-GS Variants

Table 5 summarizes the main LG-GS variants. The selected variant is LG5, a wide 518×294 , stride-1 reconstruction with 12 uniformly spaced keyframes. It exports approximately 1.83M splats, a 119 MB PLY, a 600 MB attribute tensor, and 21 WebUI chunks totaling roughly 20 MB.

Table 5: LG-GS variants. The selected version best matches LingBot RGB/depth field of view while preserving high point density.

Variant	Splats	Preprocess	Outcome
LG1 stride2	602k	early LG-GS, stride 2	too sparse; grid/gap artifacts
LG2 wide	457k	wide 518×294 , stride 2	better FOV but blurry after scale inflation
LG3 letter-box	1.37M	letterbox to square	improved density; dark splats and grid artifacts remained
LG4 clean	1.14M kept	letterbox cleanup	+ reduced black gaps; display-only cleanup could over-smooth
LG5 wide	1.83M	wide 518×294 , stride 1	selected; best FOV consistency; still needs optimization

4.9 Experiment 7: Field of View

The preprocessing experiment matters because aspect-ratio or crop errors can be mistaken for pose errors. With the default square crop, the RGB window may show the right wall or hallway edge while the Gaussian window lacks those regions. This mismatch makes the views appear unsynchronized even when the camera pose branch is correct.

The wide 518×294 route keeps the RGB preview, LingBot depth, and Gaussian initialization in the same aspect ratio. The letterbox route preserves full FOV while satisfying a square input, but padding may affect appearance prediction. For the current report, the wide path is the most reliable because its pixel-depth mapping is simpler and its FOV matches LingBot.

4.10 Experiment 8: Black Speckles, Grid Artifacts, and Blur

LG-GS improved synchronization but did not solve all visual artifacts. In the letterbox stride-1 variant, more than one million splats had opacity below 0.3. Combined with very small scale axes and black background rendering, this caused black speckles and grid-like wall gaps.

The cleanup variant dropped:

- opacity below 0.04,
- dark low-opacity splats with luminance below 0.16 and opacity below 0.18.

This reduced black gaps, but the trade-off remained:

- small Gaussian scale gives sharper details but more holes,
- large Gaussian scale fills holes but causes blur,
- aggressive opacity filtering removes noise but can remove valid surfaces.

Display calibration is useful for inspection, but final asset quality requires photometric optimization.

5 Discussion

5.1 What LG-GS Actually Fixes

LG-GS primarily fixes a geometric synchronization problem. In the raw route, GenWildSplat owns the camera gauge; in the LingBot route, the WebUI and geometry own the camera gauge. By injecting Gaussian means from LingBot depth and poses, the Gaussian scene is placed into the same world frame used by the RGB playback and point cloud. This makes RGB-vs-Gaussian comparison meaningful at the level of pose synchronization and coordinate-frame consistency.

LG-GS does not guarantee sharp visual quality by itself. If LingBot depth is wrong on a wall, the Gaussian mean is also wrong. If GenWildSplat predicts unsuitable opacity or scale, the render still has artifacts. Therefore, LG-GS is better understood as a stronger initialization, not the final optimization.

5.2 Why Local Chunks Look Better Than Global Reconstructions

Local chunks have stronger overlap and simpler geometry. The model sees a smaller, more coherent scene, so the feed-forward prior produces sharper local visual structure. Global reconstruction must cover sofa/table and hallway regions together, sometimes with weak or no direct overlap. This increases ambiguity in pose, scale, and appearance. The model then averages or stretches the scene, producing blur.

This result suggests a windowed strategy:

- use LingBot/TSDF/mesh for global continuity,
- use local optimized Gaussians for high-quality current-view rendering,

- switch or blend local visual assets using geometry-aware alignment rather than raw temporal chunks.

5.3 Why Real-Time Design Matters

The live pipeline shows that real-time behavior is a scheduling problem as much as a model problem. A robot operator or controller needs current RGB and pose quickly. Dense geometry can update at a slower rate. Gaussian reconstruction can be slower still when it is used as a visual asset generation path.

This is consistent with broader embodied AI systems: reasoning, reconstruction, mapping, and control often run asynchronously. A system that blocks fast control on slow reconstruction may be easier to analyze offline but less reliable online. Our design therefore keeps the fast path alive and treats dense geometry as latest-available state.

5.4 Evidence Summary

Because Mono2Sim-GS combines robotic perception, learned geometry, and Gaussian rendering, a single metric would be misleading. A high RGB frame rate does not imply real-time dense reconstruction, and a visually sharp Gaussian render does not imply simulator-ready geometry. We therefore interpret the experiments along three axes: system latency, geometric consistency, and visual asset quality.

Table 6 summarizes the main evidence used in this report. The most important result is the separation between local visual quality and global scene continuity. Local GenWildSplat chunks can look sharp because the input views have stronger overlap and cover a smaller spatial region. However, when the same feed-forward prior is asked to explain the entire 301-frame sequence with sparse global context views, it must infer scale, pose, depth, and appearance across weakly connected regions. This produces blur, memory pressure, and unstable global layout. LG-GS addresses the geometric part of this failure by grounding Gaussian means in LingBot depth and poses, but it does not replace the need for photometric optimization or simulator-side mesh extraction.

Table 6: Main evidence and interpretation. The table separates live throughput, geometric alignment, and visual asset quality.

Axis	Evidence	Interpretation
Live path	RGB preview runs at 4–5 fps	The robot-side loop is responsive enough for operator inspection in this prototype, but dense reconstruction is asynchronous.
Geometry	LingBot processes the 301-frame video into depth, pose, and point clouds	Provides a shared world frame for inspection and Gaussian grounding.
Local GS	Best local chunk alignment error is 0.0505; late hallway chunk rises to 0.2229	Sparse-view Gaussian priors work best in overlapping local windows.
Global GS	32–44 context views complete; 48-view global run exceeds memory	More context increases coverage but stresses memory and global coherence.
LG-GS	Selected wide stride-1 variant exports about 1.83M splats and 21 WebUI chunks	Improves pose synchronization, coordinate-frame consistency, and FOV consistency, but remains an initialization.
Artifacts	Low-opacity splats, small scales, and black-background rendering create speckles and holes	Scale/opacity regularization and photometric refinement are needed for final visual quality.

This evaluation also clarifies the project’s claim boundary. The results support four bounded claims:

- The live edge path can keep RGB, pose, and operator feedback usable for inspection while heavier reconstruction runs asynchronously.
- LingBot geometry provides a practical world-frame anchor for aligning point clouds, camera playback, and Gaussian assets.
- Feed-forward sparse-view Gaussian reconstruction is most reliable in local overlapping windows; long weakly-overlapping

robot video needs additional global geometry.

- LG-GS improves camera-gauge consistency by moving Gaussian initialization into the LingBot frame, but it remains an initialization rather than a finished optimized scene.

The results do not yet demonstrate a completed physics-ready simulator scene. In particular, a renderable Gaussian asset should not be treated as a collision mesh, and a responsive live preview should not be interpreted as per-frame dense reconstruction. A complete real-to-sim export should be evaluated with held-out RGB render metrics, surface accuracy or completeness, collision-map validity, and downstream navigation or manipulation performance.

5.5 Implications for Embodied Systems

The reconstruction layer is most useful as infrastructure for embodied systems rather than as a standalone visual demo. It can support several task families:

- **Semantic navigation:** attach object labels or language references to persistent 3D scene regions.
- **Mobile manipulation:** use geometry for obstacle avoidance and arm-base coordination.
- **Object search:** remember where objects were seen across a trajectory.
- **VLA grounding:** connect natural-language instructions to 3D visual context.
- **Digital twins:** bootstrap indoor visual assets from ordinary RGB video.
- **World-model training:** provide persistent visual scene memory for long-horizon reasoning.

These use cases motivate the separation between geometry and appearance: planning and manipulation need stable physical structure, while policy evaluation and human inspection benefit from photorealistic visual context.

6 Limitations

The current system has several important limitations.

1. **No completed physical simulator export.** TSDF fusion, mesh cleanup, collision assets, NAV2 maps, and Isaac Sim export remain future work.
2. **Monocular depth uncertainty.** LingBot depth can be inaccurate on low-texture walls, reflective boundaries, and repeated structures. LG-GS inherits those errors.
3. **Hard geometry injection.** The current LG-GS implementation directly replaces or initializes Gaussian means with LingBot-unprojected points. A softer learned conditioning mechanism may be better.
4. **Limited quantitative image metrics.** We mainly use visual inspection, point counts, latency, alignment errors, and artifact analysis. Future work should compute PSNR/SSIM/LPIPS on held-out frames after rendering with synchronized poses.
5. **Keyframe selection is still simple.** The current 12-keyframe set spans time uniformly. Motion-aware or coverage-aware selection should improve results.
6. **Display cleanup is not reconstruction.** Opacity and scale filtering improve WebUI readability but cannot replace photometric Gaussian optimization.

7 Conclusion

This project built a functional RGB-first reconstruction layer toward real-to-sim scene generation. The system connects monocular RGB input, low-latency pose, LingBot learned geometry, GenWildSplat Gaussian visual priors, and a GS Console inspection UI. The main method, LG-GS, changes GenWildSplat from

a pose-free sparse-view reconstruction component into a pose-guided visual prior by injecting LingBot depth, intrinsics, and camera poses into Gaussian mean initialization during export.

The experiments show that raw GenWildSplat is useful locally but unreliable as a long-video global mapper. Overlapping chunks preserve local sharpness but introduce switching and alignment artifacts. Single global multi-keyframe reconstructions improve coverage but often become blurry and memory-heavy. LG-GS improves pose synchronization and coordinate-frame consistency and provides a stronger initialization for a global visual reconstruction layer, but it still needs local photometric Gaussian optimization to reach final visual quality.

The main design conclusion is that, for robot RGB video, Gaussian reconstruction should be geometry-guided and system-aware, not only image-driven.

In the next stage, the project should fuse LingBot depth and pose into TSDF/mesh/occupancy assets, optimize LG-GS Gaussians photometrically in local windows, and export a complete scene bundle containing RGB, poses, depth, confidence, point cloud, mesh, Gaussian, navigation, and simulator metadata.

References

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 139:1–139:14, 2023.
- [2] A. Korovko, D. Slepichev, A. Eftorov, A. Dzhumamuratova, V. Kuznetsov, H. Rabeti, J. Biswas, and S. Pouya, “cuVSLAM: CUDA accelerated visual odometry and mapping,” 2025, arXiv:2506.04359.
- [3] L.-Z. Chen, J. Gao, Y. Chen, K. L. Cheng, Y. Sun, L. Hu, N. Xue, X. Zhu, Y. Shen, Y. Yao, and Y. Xu, “Geometric context transformer for streaming 3D reconstruction,” 2026, arXiv:2604.14141.
- [4] V. Gupta, C.-H. Lin, S. Wang, A. Bhattad, and J.-B. Huang, “Generalizable sparse-view 3D reconstruction from unconstrained images,” 2026, arXiv:2604.28193.
- [5] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion revisited,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4104–4113.
- [6] B. Curless and M. Levoy, “A volumetric method for building complex models from range images,” in *Proceedings of SIGGRAPH*, 1996, pp. 303–312.
- [7] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohli, J. Shotton, S. Hodges, and A. Fitzgibbon, “KinectFusion: Real-time dense surface mapping and tracking,” in *IEEE International Symposium on Mixed and Augmented Reality*, 2011, pp. 127–136.
- [8] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *European Conference on Computer Vision*, 2020, pp. 405–421.
- [9] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, “SplaTAM: Splat, track and map 3D gaussians for dense RGB-D SLAM,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 357–21 366.
- [10] D. Li, A. Yadav, C. Peng, R. Chellappa, and A. Bhattad, “SyncFix: Fixing 3D reconstructions via multi-view synchronization,” 2026, arXiv:2604.11797.
- [11] Z. Huang, Y. Zhang, J. Liu, R. Song, C. Tang, and J. Ma, “TIC-VLA: A think-in-control vision-language-action model for robot navigation in dynamic environments,” 2026, arXiv:2602.02459.